

Entorno Linux, Shell y reproducibilidad

TC-CH01 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. ¿Qué ocurre exactamente entre que teclea una orden y obtiene un resultado, y cómo consigue que otra persona pueda reproducir ese resultado en su propia máquina?

Producto final. Un espacio de trabajo Linux explorado sin alterarlo, con el entorno registrado, las operaciones peligrosas comprobadas antes de ejecutarlas y una entrega empaquetada cuya integridad se verifica con sha256sum.

Mapa del tema

1. Antes de la primera orden	1
2. Unix y sus paradigmas	2
3. El modelo terminal, Shell y procesos	4
4. El sistema de archivos: el mapa de navegación	6
5. Navegación e inspección sin alterar	8
6. Enlaces duros y enlaces simbólicos	10
7. Propiedad y permisos	10
8. Ejecutar un archivo y variables de entorno	12
9. Ergonomía y productividad en la terminal	13
10. Espacio de almacenamiento y organización del proyecto	14
11. Antes de una operación que sobrescribe o elimina	14
12. Reproducibilidad, empaquetado e integridad	15
13. Método de verificación y errores frecuentes	15
14. Síntesis de referencia	16
15. Glosario esencial	17
16. Fuentes y límites de uso	17

Cómo usar este tema

Este capítulo se lee antes de la primera sesión de laboratorio y se trabaja en tres tiempos: primero el modelo (qué es un proceso, qué es una ruta, qué es un permiso), después las órdenes que lo hacen visible, y por último el anexo, donde todo se ejecuta sobre un corpus pequeño. Las cuatro predicciones de la sección siguiente conviene responderlas por escrito antes de leer: el capítulo vuelve sobre cada una.

Al terminar sabrá: orientarse en el sistema de archivos con rutas absolutas y relativas; inspeccionar un fichero grande sin cargarlo entero; leer y cambiar permisos; explicar por qué una orden se encuentra o no se encuentra; registrar su entorno de trabajo; y empaquetar y verificar una entrega.

Las marcas **ESENCIAL**, **ESTUDIO** y **REFERENCIA** indican qué bloques forman la ruta esencial (asignable antes de clase), cuáles amplían y cuáles son de consulta. La ruta esencial comprende la filosofía Unix, el modelo de procesos, el mapa del sistema de archivos, la navegación e inspección segura, los permisos, los enlaces y la ejecución de programas. La ruta de estudio añade variables de entorno, personalización, ergonomía, almacenamiento, registro de entorno, protocolo de seguridad y empaquetado. Los bloques de terminal se escriben sin el símbolo del *prompt*; ejecute cada uno en una carpeta de trabajo, prediga la salida y compárela.

1 Antes de la primera orden

El análisis masivo de datos biológicos rara vez cabe en una hoja de cálculo. Cuando un secuenciador entrega decenas de gigabytes con millones de lecturas, o un modelo estructural produce miles de conformaciones, una interfaz gráfica deja de ser práctica: cargar el fichero entero satura la memoria, y repetir la misma operación sobre cientos de muestras a mano es inviable. La terminal procesa los datos por partes y permite automatizar la operación de forma reproducible.

La terminal no es un entorno hostil diseñado para programadores expertos; es una conversación directa con el sistema operativo. En vez de hacer clic sobre botones que ocultan lo que ocurre por debajo, usted escribe

instrucciones precisas que transforman flujos de información biológica de forma transparente y automatizable.

Sin embargo, en biomedicina y bioinformática, obtener un número o una tabla no basta. La ciencia exige que cualquier análisis ejecutado hoy pueda ser repetido por otra persona, o por usted mismo seis meses más tarde, con el mismo procedimiento y un criterio declarado para comparar el resultado. A esta exigencia la llamamos **reproducibilidad computacional**.

Este capítulo establece los cimientos de todo su trabajo técnico posterior. Aprenderá cómo está construido el ecosistema Unix y Linux; qué ocurre en la memoria del ordenador cuando se ejecuta un programa; cómo orientarse en el sistema de archivos sin perderse; cómo inspeccionar ficheros biológicos masivos sin alterarlos; cómo gestionar permisos y enlaces; cómo personalizar su entorno de trabajo; y cómo registrar un experimento computacional para que sea verificable.

Antes de empezar: cuatro predicciones formativas.

ESENCIAL Sin ejecutar ninguna orden en la terminal, anote su predicción razonada ante estas cuatro situaciones:

1. ¿Designa siempre la expresión `datos/muestra.fasta` exactamente el mismo fichero?
2. ¿Un fichero con permiso de lectura es obligatoriamente ejecutable por el usuario?
3. ¿Un estado de salida 0 (`$? = 0`) certifica que el resultado científico del cálculo es correcto?
4. ¿Una variable creada en la sesión actual de Bash llega automáticamente a un subprocesso hijo?

Guarde sus respuestas. Las fundamentaremos y resolveremos progresivamente en las secciones 4 (rutas), 7 (permisos), 3 (estado de salida) y 8 (variables de entorno).

2 Unix y sus paradigmas

2.1 Un poco de historia: el origen de la computación moderna

En 1969, en los laboratorios Bell de AT&T, Ken Thompson y Dennis Ritchie comenzaron a diseñar un sistema operativo interactivo, multitarea y multiusuario al que bautizaron como Unix. Poco después, Ritchie diseñó el lenguaje C para programación de sistemas, y en 1973 Unix se reescribió en C. Esa decisión lo hizo portable a hardware muy diverso: hasta entonces, los sistemas operativos estaban atados a la máquina para la que se programaban.

A partir de 1974, AT&T licenció Unix a universidades y centros de investigación a precios simbólicos. En la Universidad de California en Berkeley nació una influyente variante académica conocida como BSD (*Berkeley Software Distribution*). En 1983, Richard Stallman fundó el proyecto GNU con el objetivo de construir un sistema operativo completamente libre y compatible con Unix. En 1991, un estudiante finlandés llamado Linus Torvalds desarrolló el núcleo (*kernel*) que faltaba para completar ese sistema: nacía Linux. Por su parte, la familia macOS de Apple deriva de NeXTSTEP, creado en 1988 tras la salida de Steve Jobs de Apple; Apple adquirió NeXT en 1996 y, sobre esa base, Mac OS X (2001) incorporó el núcleo Darwin, de raíz BSD y Mach.

Unix se desarrolló en los laboratorios Bell de AT&T en 1969 (Ken Thompson y Dennis Ritchie); Linux y macOS son sistemas derivados o basados en Unix, no el propio Unix original.

«Unix» y «tipo Unix» (*Unix-like*) no son lo mismo: Linux y macOS clonan o derivan el funcionamiento central de Unix sin ser Unix en sentido estricto.

Esta genealogía no es una anécdota histórica: es la razón de que los conocimientos de línea de órdenes y Bash que aprenderá en este libro se transfieran en gran medida a un portátil con Linux, a un MacBook o al nodo de un superordenador. El detalle cambia: macOS abre `zsh` por defecto y muchas de sus utilidades provienen de BSD, no de GNU, por lo que algunas opciones difieren. Cuando una orden se comporte distinto, la explicación suele estar ahí.

2.2 Por qué importa: presencia profesional e infraestructura científica

En la actualidad, los sistemas basados en Linux constituyen el estándar predominante de la computación científica y de la infraestructura informática mundial. No se trata de una preferencia estética, sino de una

realidad operativa.

Los sistemas basados en Linux constituyen el estándar predominante en servidores, infraestructuras de cálculo científico en la nube y supercomputación, proporcionando el entorno operativo homogéneo donde se ejecutan los análisis bioinformáticos modernos.

La conclusión didáctica no depende de una cuota estadística puntual: cuando descargue datos del NCBI o de Ensembl, ejecute un modelo estructural, alinee millones de lecturas en el clúster de un hospital o lance un contenedor reproducible, casi siempre estará trabajando sobre un sistema Linux o compatible. Por eso lo que aprende aquí se reutiliza en todos esos contextos.

2.3 Distribuciones: el mismo núcleo, distintas maneras de empaquetarlo

ESTUDIO Linux es el núcleo; una **distribución** añade el gestor de paquetes, las utilidades y la política de actualizaciones. Debian y sus derivadas (Ubuntu, Mint) usan paquetes `.deb` y la orden `apt`; Fedora, RHEL y sus derivadas (Rocky, Alma) usan `.rpm` y `dnf`. En un clúster científico encontrará casi siempre una de estas dos familias, y las diferencias que notará son de gestor de paquetes y de versiones, no de Shell. Cómo probar o instalar Ubuntu en su portátil se explica en la guía de laboratorio y en la documentación de Canonical [4].

2.4 Filosofía de código abierto

El software de código abierto (*open source*) permite que cualquiera vea, modifique y redistribuya el código; su desarrollo suele ser descentralizado y colaborativo, con comunidades que revisan el código públicamente en repositorios.

En biomedicina, la ciencia abierta es un imperativo ético y metodológico. Si un algoritmo calcula una dosis terapéutica o predice la patogenicidad de una variante genética, la comunidad debe poder auditar el código fuente para entender su mecanismo y descartar artefactos. Conviene distinguir tres cosas que a menudo se confunden: que el código sea *visible* (se puede leer), que sea *libre* (se puede modificar y usar sin restricción) y que sea *redistribuible* (se puede compartir la copia modificada). Linux, Bash y las utilidades que empleamos en este curso cumplen las tres.

2.5 Primer paradigma: todo es un fichero

En el diseño clásico de Unix existe un principio unificador de enorme elegancia: *todo es un fichero*.

En los sistemas Unix, la mayoría de los recursos se exponen como ficheros con una interfaz común de lectura y escritura: discos, dispositivos, tuberías, incluso información del núcleo en `/proc`.

Es un principio de diseño de la interfaz, no una identidad literal: la memoria o la pantalla no se editan como un texto. Lo que significa es que, para el sistema operativo, no hay una diferencia conceptual abismal entre leer una línea de un fichero FASTA guardado en el disco, recibir bytes de un secuenciador a través de la red o leer los datos que otro programa le envía en tiempo real: todos se presentan como una corriente de bytes que puede abrirse, leerse, transformarse y cerrarse con las mismas órdenes básicas.

2.6 Segundo paradigma: herramientas pequeñas que se combinan

Frente a la tentación de construir programas gigantescos que intenten resolver todas las tareas posibles, y que acaban siendo frágiles y difíciles de mantener, Unix apostó por una arquitectura modular: herramientas pequeñas y robustas, cada una dedicada a una sola cosa, con mecanismos estándar para conectarse entre sí.

Un ejemplo clásico es `tar`: su única función es empaquetar múltiples ficheros dentro de un único contenedor sin comprimir. Cuando se necesita comprimir, Unix no reescribió `tar` para convertirlo en un compresor; utilizó `gzip`, un programa especializado en comprimir flujos de bytes [7].

El diseño de `tar -z`, que integra la compresión `gzip` en el proceso de archivado sin fusionar ambos programas en uno, ejemplifica la filosofía Unix de herramientas pequeñas diseñadas para trabajar juntas, no de un programa único con todas las funciones.

En el capítulo siguiente aprenderá a encadenar estas herramientas mediante tuberías (`|`), construyendo auténticas cadenas de montaje de datos biológicos.

3 El modelo terminal, Shell y procesos

Para diagnosticar con éxito cualquier situación en la línea de órdenes, es imprescindible distinguir tres elementos que los principiantes suelen confundir:

Una terminal es una interfaz de texto que envía órdenes a una Shell, la cual las interpreta y, cuando procede, lanza procesos; el resultado observable de un proceso incluye su salida y un estado de salida entero.

- **La terminal:** es la ventana (el emulador) que captura lo que usted teclea y dibuja los caracteres que el sistema devuelve.
- **La Shell:** es el programa que lee las líneas que usted escribe, analiza su sintaxis, busca los ejecutables correspondientes y pide al sistema operativo su ejecución. En este libro utilizamos **Bash** (*Bourne Again Shell*) [5].
- **El proceso:** es la instancia viva de un programa bajo control del sistema operativo: ocupa memoria propia y, según su estado, se ejecuta en la CPU o espera (un dato del disco, una señal, un turno de planificación).

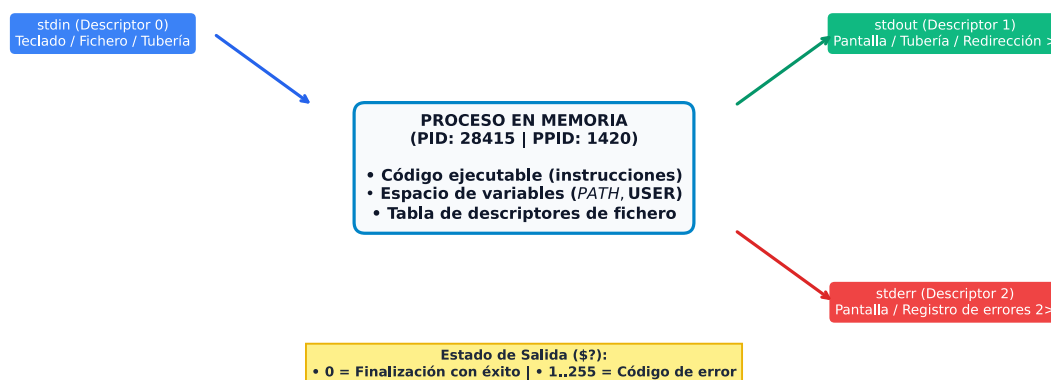


Figura 1: Modelo de proceso en memoria y descriptores estándar de entrada y salida (stdin, stdout, stderr) bajo el sistema operativo. Figura original adaptada de las presentaciones docentes de la asignatura; las redirecciones se estudian en el capítulo siguiente.

3.1 Qué escribe usted: la anatomía de una orden

ESENCIAL Cada línea que teclea tiene una estructura fija que la Shell analiza de izquierda a derecha. Tome como ejemplo esta línea tal como aparece en pantalla:

```
alumno@servidor:~/TC01$ ls -l -h datos/ > listado.txt
```

- **Prompt** (alumno@servidor:~/TC01\$): lo dibuja la Shell; no se teclea. Suele indicar usuario, máquina y directorio de trabajo.
- **Orden** (ls): el programa que se ejecuta; el verbo de la frase.
- **Opciones** (-l, -h): modifican el comportamiento; empiezan por - en su forma corta o por -- en la larga.
- **Argumentos** (datos/): los datos sobre los que actúa la orden, normalmente rutas.
- **Redirección** (> listado.txt): a dónde va la salida; se estudia en el capítulo siguiente.

Separar estas piezas es lo que permite leer una orden ajena, o una que le proponga un asistente, y predecir qué hará antes de ejecutarla.

3.2 De un texto a un proceso

ESENCIAL El programa que usted escribe, o que trae el sistema, es en el disco un fichero: texto legible o código ya traducido. Para que la CPU lo ejecute, su contenido debe convertirse en instrucciones de **código máquina**, números que la CPU interpreta como sumar, guardar o saltar. Esa traducción la hace un compilador, una vez y de antemano, o un intérprete, sobre la marcha, como hace Bash con un script. Cuando lanza la orden, el sistema operativo carga esas instrucciones en la memoria, les asigna un identificador (PID) y un turno de CPU, y nace el **proceso**: el programa deja de ser un fichero inerte y pasa a consumir recursos y a transitar por los estados que siguen.

3.3 El ciclo de vida de un proceso

Un proceso no es un bloque inerte. Desde que nace hasta que termina, transita por diversos estados:

Un proceso Unix transita durante su vida por estados bien definidos, en ejecución, en espera, detenido y, en casos patológicos, zombi; no es una entidad estática mientras dura.

- **En ejecución** (*running*): el proceso está ejecutando instrucciones en la CPU o esperando su turno inmediato en la cola del planificador.
- **En espera** (*sleeping*): el proceso aguarda que ocurra un evento externo, por ejemplo que el disco termine de leer un fichero de secuencias o que llegue un paquete de red. Una espera puede ser interrumpible por una señal o no serlo, según lo que espere.
- **Detenido** (*stopped*): el proceso ha sido pausado temporalmente, por ejemplo mediante una señal del usuario para depuración.
- **Zombi**: el proceso ha finalizado pero su padre aún no ha leído su estado de terminación. Apenas consume recursos, pero conserva su entrada en la tabla de procesos y ocupa un PID.

Los cambios de estado provocados desde fuera se hacen con **señales**: SIGINT es la que envía Ctrl+C y pide interrumpir; SIGTERM pide terminar de forma ordenada; SIGKILL obliga al sistema a eliminar el proceso sin que este pueda atenderla.

3.4 Jerarquía y aislamiento de memoria

En Linux, todos los procesos forman un árbol cuya raíz es el proceso primordial del sistema (`init` o `systemd`, con PID 1).

Los procesos de un sistema Linux se organizan en un árbol con `init` como raíz; cada proceso hijo obtenido con `fork()` recibe un PID distinto del de su padre pero hereda sus valores iniciales.

Para crear un nuevo proceso, el sistema operativo utiliza la llamada al sistema `fork()`, que duplica el proceso actual generando un hijo con su propio identificador. El hijo suele invocar a continuación `exec()` para sustituir su imagen por el programa pedido: así lanza la Shell cada orden que usted teclea.

Cada proceso Unix tiene un espacio de memoria aislado del de los demás; la comunicación entre procesos requiere un mecanismo explícito (señales, tuberías o memoria compartida gestionada por el núcleo), no ocurre por defecto.

Este aislamiento sostiene la estabilidad de los servidores científicos: si un alineador de secuencias comete un error de segmentación o agota su memoria, normalmente solo muere ese proceso, sin corromper la memoria de la Shell ni de otros procesos. Agotar la memoria o el disco del nodo entero sí puede afectar a los demás: por eso los clústeres imponen límites por trabajo.

3.5 Privilegios: usuarios y superusuario

El superusuario de un sistema Linux se llama `root` y tiene UID 0; un usuario sin privilegios puede escalar temporalmente su nivel de privilegio solo si pertenece a un grupo autorizado para ello, y debe hacerlo de forma explícita, no por defecto.

En un entorno de cálculo compartido, los usuarios ordinarios tienen permiso para crear, modificar y borrar

exclusivamente sus propios ficheros. Solo el administrador (root) o usuarios autorizados mediante sudo pueden alterar ficheros del sistema o instalar software global. El hardware colabora en esta separación: la CPU distingue un modo privilegiado, reservado al núcleo, del modo en que corren los programas de usuario. Un usuario normal solo escala a root mediante sudo, y únicamente si pertenece a un grupo autorizado (sudo, wheel o admin, según la distribución).

3.6 El estado de salida de un proceso

Cuando un proceso finaliza, devuelve al sistema operativo un número entero entre 0 y 255 conocido como **estado de salida** (*exit status*). En Bash, este valor queda registrado en la variable especial \$?.

El estado de salida de un proceso en Bash queda disponible en \$? inmediatamente después de su ejecución, y vale 0 por convención cuando el proceso no detectó error; no certifica que el resultado sea correcto, solo que el proceso terminó de cierta manera.

Un estado de salida 0 indica que el programa terminó sin detectar errores internos. Un valor distinto de 0 indica una condición que el propio programa define: a menudo es un error (1, 2, 127 son valores frecuentes), pero puede ser un resultado esperado. grep devuelve 1 cuando no encuentra coincidencias, y eso no es un fallo. Esta convención es la base que permite a los flujos automatizados decidir si continúan con el siguiente paso o se detienen.

Se reproduce con `verification/chapter_results.sh exit_status`.

Resolución de la predicción 3.

El estado de salida 0 solo certifica que el proceso finalizó conforme a las convenciones del sistema operativo sin abortar por error técnico. No valida la veracidad, calidad ni adecuación del resultado biológico; la comprobación científica requiere siempre la inspección explícita de lo que se ha producido.

4 El sistema de archivos: el mapa de navegación

A diferencia de otros sistemas operativos que asignan letras independientes a cada disco o partición (C:, D:), Linux organiza todo su almacenamiento en un **único árbol jerárquico** que parte de una raíz común representada por una barra: /. Todo lo que existe en el sistema, discos locales, discos de red, memorias USB, ficheros de configuración y datos, se encuentra en alguna rama de ese único árbol.

4.1 Directorio de trabajo: el suelo donde pisamos

Cada sesión de Shell y cada proceso tiene en todo momento un **directorio de trabajo** (*working directory*): es la posición actual dentro del árbol desde la cual se interpretan todas las rutas relativas.

- **Ruta absoluta:** la dirección completa de un fichero desde la raíz. Comienza siempre por /, por ejemplo /home/alumno/TC01/datos/secuencias.fasta. Es válida sin importar en qué carpeta se encuentre usted.
- **Ruta relativa:** una dirección calculada a partir del directorio de trabajo. No empieza por /, por ejemplo datos/secuencias.fasta.

Una ruta relativa se interpreta respecto al directorio de trabajo del proceso que la evalúa; la misma cadena de texto puede referirse a ficheros distintos según ese directorio.

Resolución de la predicción 1.

La expresión datos/muestra.fasta es una ruta relativa: su significado cambia según cuál sea el directorio de trabajo del proceso en cada momento. Solo una ruta absoluta identifica la misma ubicación desde cualquier sitio.

La mayoría de los errores de las primeras semanas (el mensaje No such file or directory) ocurren porque quien trabaja asume que está en una carpeta distinta de la real y usa una ruta relativa que no coincide con su posición en el árbol.

Para el árbol de ejemplo, la ruta relativa datos/tc01_mini.fasta solo resuelve al fichero real cuando el

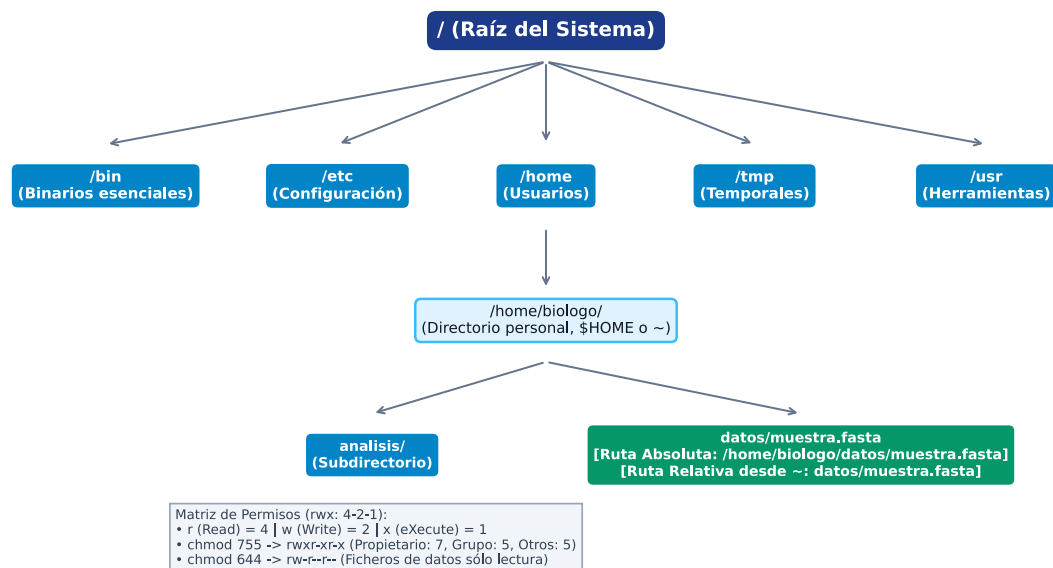


Figura 2: Jerarquía en árbol de raíz única (/), resolución de rutas absolutas frente a relativas y matriz octal de permisos (rwx). Figura original adaptada de las presentaciones docentes de la asignatura; la parte derecha se usa en la sección de permisos.

	/
	home/
	alumno/
	TC01/
	datos/tc01_mini.fasta
Directorio de trabajo	datos/tc01_mini.fasta señala a
/home/alumno/TC01	/home/alumno/TC01/datos/tc01_mini.fasta (existe)
/home/alumno	/home/alumno/datos/tc01_mini.fasta (no existe)

Figura 3: La misma ruta relativa apunta a ficheros distintos, o a ninguno, según el directorio de trabajo. Figura original.

directorio de trabajo es `/home/alumno/TC01`; desde `/home/alumno` señala a una ruta inexistente.

4.2 La brújula de la terminal

Para moverse con agilidad y escribir rutas sin redundancia, la Shell dispone de referencias estándar:

- `.` (punto): el directorio actual.
- `..` (dos puntos): el directorio padre, un nivel por encima.
- `~` (virgulilla): el directorio personal del usuario, habitualmente `/home/nombre_usuario`.
- `pwd`: imprime la ruta absoluta del directorio de trabajo [3, pp. 51–54].

Tras `mkdir -p a/b/c && cd a/b/c`, la ruta relativa `../..` conduce al directorio `a`; `pwd` tras ambas operaciones lo confirma.

Se reproduce con `verification/chapter_results.sh relative_path`.

5 Navegación e inspección sin alterar

El trabajo cotidiano en la línea de órdenes sigue una regla de oro: **inspeccionar antes de modificar**. Modificar o borrar a ciegas es la causa principal de pérdidas irreparables de datos experimentales. Las órdenes de esta tabla están documentadas en el manual de GNU Coreutils [6] y, de forma compacta, en la guía de bolsillo de Barrett [2].

Orden	Efecto	Nota de control
<code>pwd</code>	Imprime el directorio de trabajo absoluto	La primera orden ante cualquier duda de ubicación.
<code>ls</code>	Lista los ficheros del directorio indicado o actual	<code>ls -l</code> añade permisos, propietario y tamaño; <code>ls -la</code> muestra ocultos; <code>ls -lh</code> muestra tamaños legibles.
<code>cd RUTA</code>	Cambia el directorio de trabajo	<code>cd</code> solo vuelve al directorio personal; <code>cd -</code> regresa al anterior; <code>cd ..</code> sube un nivel.
<code>mkdir -p RUTA</code>	Crea directorios, intermedios incluidos	<code>-p</code> evita errores si los padres no existen o la carpeta ya existía.
<code>cp ORIGEN DESTINO</code>	Copia ficheros o carpetas (<code>cp -r</code>)	Conserva el original; confirme el destino antes de sobrescribir.
<code>mv ORIGEN DESTINO</code>	Mueve o renombra	No deja copia en el origen; en el mismo disco es instantáneo.
<code>touch FICHERO</code>	Crea un fichero vacío o actualiza su fecha	No altera el contenido si ya existía.
<code>file FICHERO</code>	Determina la naturaleza real del fichero	Analiza los bytes de cabecera, no la extensión.
<code>head / tail</code>	Muestra las primeras o últimas líneas	Por defecto 10; <code>-n NUM</code> fija la cantidad.
<code>cat / tac / more</code>	Vuelca un fichero entero, del revés, o por páginas	<code>cat</code> solo para ficheros pequeños; <code>tac</code> invierte el orden de líneas; <code>more</code> es un paginador anterior a <code>less</code> .
<code>less FICHERO</code>	Visor interactivo paginado	Explora ficheros de gigabytes sin cargarlos enteros; se sale con <code>q</code> .
<code>wc FICHERO</code>	Cuenta líneas (<code>-l</code>), palabras (<code>-w</code>) y bytes (<code>-c</code>)	<code>wc -l</code> cuenta saltos de línea, no entidades biológicas.

Orden	Efecto	Nota de control
<code>basename / dirname</code>	Separan el nombre del fichero y su directorio en una ruta	Útiles en scripts para construir nombres de salida.

`pwd`, `ls`, `cd`, `mkdir -p`, `cp`, `mv`, `touch` y las órdenes de inspección (`file`, `head`, `tail`, `less`, `wc`) son las órdenes mínimas de navegación, organización e inspección documentadas en GNU Coreutils.

La comparación de ficheros con `diff` y la búsqueda con `find` se estudian en el capítulo siguiente, junto con las herramientas de texto.

5.1 Inspección segura de datos biológicos masivos

Uno de los errores más destructivos al comenzar es hacer doble clic sobre un fichero de secuenciación de 15 GB para «abrirlo» con un editor gráfico. El programa intentará cargar los 15 gigabytes íntegros en la memoria y el sistema dejará de responder. Las utilidades de Unix están diseñadas para manejar flujos de datos sin importar su tamaño:

- `head -n 20 lecturas.fastq` lee únicamente las primeras 20 líneas y se detiene, consumiendo unos pocos kilobytes.
- `tail -n 20 lecturas.fastq` muestra las últimas 20 líneas; útil para comprobar si una descarga o una simulación terminó.
- `less lecturas.fastq` abre un visor que lee el fichero por bloques a medida que usted avanza. Se desplaza con la barra espaciadora, retrocede con `b`, busca con `/patrón` y navega entre coincidencias con `n` y `N`, y sale con `q`. La misma tecla `q` cierra las páginas de `man`.

5.2 La trampa de contar líneas frente a contar secuencias

`file` identifica el tipo de un fichero a partir de su contenido, no de su extensión; `data/tc01_sample.fasta` tiene 19 líneas, verificable con `wc -l` y a mano.

La orden `wc -l` cuenta estrictamente caracteres de fin de línea. En un fichero FASTA donde cada secuencia puede ocupar varias líneas, devuelve el número de líneas del fichero, no el número de secuencias. Para contar secuencias hay que filtrar las cabeceras, como aprenderá en el capítulo siguiente con `grep -c '^>'`.

Se reproduce con `verification/chapter_results.sh file_type`.

Se reproduce con `verification/chapter_results.sh line_count_sample`.

5.3 El problema silencioso de los saltos de línea de Windows

Un problema frecuente ocurre cuando un fichero de texto se exporta desde una hoja de cálculo en Windows y luego se procesa en un servidor Linux. Linux y Unix representan el fin de línea con un único carácter, *line feed* (LF); Windows utiliza dos, *carriage return* seguido de *line feed* (CRLF). La orden `file` analiza los primeros bytes y detecta la discrepancia.

`file` detecta terminadores de línea de DOS y Windows (CRLF) distinguiéndolos de los nativos de Unix (LF), lo que evita fallos sutiles en scripts al importar datos.

Se reproduce con `verification/chapter_results.sh crlf_detection`.

5.4 Ficheros ocultos y configuración

Cualquier fichero o carpeta cuyo nombre comience por un punto (`.bashrc`, `.gitignore`, la carpeta `.git`) se considera **oculto**. Suelen contener configuraciones o metadatos, y `ls` los omite por defecto.

Los ficheros y directorios ocultos que comienzan por un punto se omiten en la salida estándar de `ls` y se listan explícitamente con `ls -a` o `ls -la`.

Se reproduce con `verification/chapter_results.sh hidden_files_listing`.

5.5 La verdad de su sistema: `--help` y `man`

Nunca confíe ciegamente en ejemplos copiados de internet sin contrastar las opciones de la herramienta en su propia instalación. Distintas distribuciones y versiones tienen parámetros ligeramente distintos.

`--help` y `man ORDEN` documentan la versión de la orden realmente instalada; un ejemplo copiado de otra fuente puede corresponder a otra versión o a otra utilidad de nombre igual.

6 Enlaces duros y enlaces simbólicos

En el sistema de archivos de Linux, la información de un fichero se divide en dos partes: los bloques de datos en el disco y una estructura de metadatos denominada **inodo**. El inodo guarda el tamaño, los permisos, las fechas y los punteros a los bloques, pero **no** guarda el nombre. El nombre es una entrada en una tabla de directorio que apunta a un número de inodo. Esta arquitectura permite dos tipos de enlace:

- **Enlace duro** (`ln origen enlace`): una nueva entrada de directorio que apunta al **mismo inodo**. Ambos nombres tienen la misma categoría; los datos permanecen mientras quede al menos un enlace. No pueden crearse entre sistemas de archivos distintos ni sobre directorios.
- **Enlace simbólico** (`ln -s origen enlace`): un fichero especial con inodo propio cuyo contenido es la ruta del destino. Borrar el enlace no afecta al original; borrar o mover el original deja el enlace «roto».

Un enlace simbólico `ln -s` crea un puntero de ruta independiente cuya eliminación deja el fichero original intacto, mientras que un enlace duro `ln` comparte el mismo inodo y el mismo contador de enlaces.

Se reproduce con `verification/chapter_results.sh symlink_creates_pointer`.

6.1 El valor de los enlaces simbólicos en bioinformática

En proyectos de genómica es habitual disponer de un genoma de referencia (como `GRCh38.fa`, que supera los 3 GB) o colecciones de lecturas almacenadas en un disco central del laboratorio.

En bioinformática, los enlaces simbólicos permiten que múltiples proyectos independientes referencien conjuntos de datos masivos compartidos sin duplicar almacenamiento físico.

En lugar de copiar 3 GB en la carpeta de cada estudiante, lo que multiplicaría el consumo de disco, se crea un enlace simbólico `ln -s /datos/referencias/GRCh38.fa ./genoma_ref.fa`. El software procesa el enlace igual que el fichero original, y el almacenamiento adicional es de unos pocos bytes.

7 Propiedad y permisos

La seguridad y la colaboración en un entorno multiusuario se basan en el modelo de permisos de Unix. Cada fichero y carpeta tiene un **usuario propietario** (`u`) y un **grupo propietario** (`g`), y define permisos independientes para tres categorías: el propietario, los miembros del grupo y todos los demás (`o`).

Para cada categoría existen tres permisos: lectura (`r`), escritura (`w`) y ejecución (`x`). La orden `ls -l` los muestra en una cadena de diez caracteres, por ejemplo `-rwxr-xr--`: el primer carácter indica el tipo de fichero y los nueve restantes son los permisos, de tres en tres, para usuario, grupo y otros.

Primer carácter	Tipo de fichero
-	fichero regular
d	directorio
l	enlace simbólico
c	dispositivo de caracteres (una terminal, <code>/dev/null</code>)
b	dispositivo de bloques (un disco)
p	tubería con nombre
s	socket

Los cuatro últimos tipos son la forma concreta en que «todo es un fichero»: un disco o una terminal aparecen en

el árbol con un nombre y unos permisos, aunque no contengan texto.

Cada fichero Unix tiene un permiso de lectura, escritura y ejecución independiente para su propietario, su grupo y el resto de usuarios; `ls -l` muestra los nueve bits resultantes en ese orden.

7.1 El significado especial de la ejecución en directorios

En un fichero ordinario, el bit `x` otorga permiso para ejecutarlo como programa. En un **directorio**, `x` significa **permiso de paso**: entrar con `cd`, atravesarlo para alcanzar subcarpetas o acceder a los inodos de los ficheros que contiene. Un directorio con `r` pero sin `x` deja ver la lista de nombres, pero no leer su contenido ni sus tamaños.

7.2 Modificación de permisos con `chmod`

La orden `chmod` (*change mode*) admite dos notaciones:

Forma	Ejemplo	Efecto
Simbólica	<code>chmod u+x script.sh</code>	Añade ejecución solo al propietario.
Simbólica	<code>chmod g-w,o-r datos/</code>	Retira escritura al grupo y lectura a los demás.
Octal	<code>chmod 751 script.sh</code>	<code>r=4, w=2, x=1</code> sumados por categoría: usuario 7 (<code>rwX</code>), grupo 5 (<code>r-x</code>), otros 1 (<code>--x</code>).
Octal	<code>chmod 644 tabla.tsv</code>	Permisos habituales de datos: usuario 6, grupo 4, otros 4.
Octal	<code>chmod 755 proyecto/</code>	Permisos habituales de carpetas y programas públicos.
Propiedad	<code>chown usuario:grupo fichero</code>	Cambia propietario y grupo. Cambiar el propietario requiere privilegios de <code>root</code> : un usuario normal no puede ceder un fichero a otro.

`chmod` admite forma simbólica (`u+x, g-w`) y forma numérica (suma de `r=4, w=2, x=1` por categoría, por ejemplo 751); `chown` cambia propietario y grupo.

Se reproduce con `verification/chapter_results.sh chmod_numeric_matches_symbolic`.

Diagnóstico de acceso en dos fases.

REFERENCIA Ante un fallo de acceso, el sistema resuelve primero la ruta y después los permisos, en este orden. La tabla resume qué comprobar en cada fase.

Mensaje	Fase y causa	Comprobación
No such file or directory	la ruta no resuelve: directorio de trabajo distinto del supuesto, errata o fichero inexistente	<code>pwd; ls -ld -- RUTA</code> sobre cada tramo
Permission denied al entrar	falta <code>x</code> en algún directorio del camino	<code>ls -ld</code> sobre cada directorio ancestro
Permission denied al leer o escribir	falta <code>r</code> o <code>w</code> en el fichero, o el sistema de archivos es de solo lectura	<code>ls -l FICHERO; df</code>
Permission denied al ejecutar	falta <code>x</code> en el fichero	<code>ls -l; chmod u+x</code> solo sobre scripts propios revisados

Resolución de la predicción 2.

Tener permiso de lectura permite examinar los bytes de un script o de un binario, pero no ejecutarlo como programa si carece del bit `x`. Los tres permisos son independientes.

7.3 Un cuarto grupo de bits: SUID, SGID y sticky

ESTUDIO Además de los nueve bits `rwX`, un fichero puede llevar tres bits especiales que cambian *quién* actúa cuando se usa.

- **SUID** (`chmod u+s, octal 4xxx`): al ejecutar el programa, el proceso adopta el usuario *propietario* del fichero, no el que lo lanzó. Así `passwd` puede modificar `/etc/shadow` aunque lo invoque un usuario normal. `ls -l` muestra una `s` en la posición de la `x` del propietario: `-rwsr-xr-x`.
- **SGID** (`chmod g+s, 2xxx`): en un ejecutable, el proceso adopta el *grupo* del fichero; en un directorio, los ficheros creados dentro heredan su grupo, lo que resuelve las carpetas de proyecto compartidas.
- **Sticky bit** (`chmod +t, 1xxx`): en un directorio de escritura común como `/tmp`, impide que un usuario borre ficheros de otro aunque tenga permiso de escritura en la carpeta. `ls -l` lo muestra como `t` final: `drwxrwxrwt`.

Atención. Un ejecutable con SUID de `root` y un fallo de programación es una vía clásica de escalado de privilegios. Nunca active SUID en un script propio.

8 Ejecutar un archivo y variables de entorno

Para que un fichero pueda ejecutarse como programa deben cumplirse dos condiciones a la vez: la Shell debe saber dónde encontrarlo, y el fichero debe tener activo el bit `x` para el usuario que lo invoca.

Ejecutar un archivo exige que la Shell pueda localizarlo (por `$PATH` o ruta explícita) y que tenga permiso de ejecución para el usuario que lo invoca; ninguna de las dos condiciones es opcional.

Si intenta ejecutar un fichero que carece del bit `x`, Bash responderá `Permission denied`.

8.1 La variable `$PATH` frente a la ruta explícita `./`

Cuando escribe el nombre de una orden sin barras (`ls`, `fastqc`), Bash no inspecciona todo el disco. Consulta la variable de entorno `$PATH`, una lista ordenada de directorios separados por dos puntos, por ejemplo `/usr/local/bin:/usr/bin:/bin`, y ejecuta el primer fichero coincidente que encuentre recorriéndola de izquierda a derecha.

Por seguridad, el directorio actual (`.`) no suele estar en `$PATH`. Si escribe un script `analisis.sh` en su carpeta y teclea `analisis.sh`, Bash dirá `command not found`. Para ejecutar el fichero de la carpeta actual sin buscar en `$PATH`, se antepone una ruta explícita: `./analisis.sh`.

Cuando se invoca un ejecutable por nombre, Bash lo busca en los directorios listados en `$PATH`, en orden; una ruta explícita omite esa búsqueda.

Se reproduce con `verification/chapter_results.sh path_resolution`.

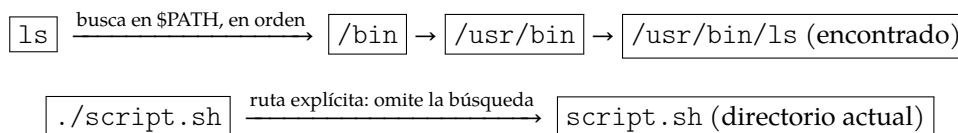


Figura 4: Arriba, el nombre `ls` se busca en cada directorio de `$PATH` hasta encontrar `/usr/bin/ls`; abajo, la ruta explícita `./script.sh` evita esa búsqueda y usa el fichero del directorio actual. Figura original.

Localizar un ejecutable por nombre recorre `$PATH` en orden hasta encontrarlo; una ruta explícita, por ejemplo `./script.sh`, omite esa búsqueda y usa directamente el fichero indicado.

8.2 El shebang: el contrato de ejecución

La primera línea de un script ejecutable contiene el **shebang**:

```
#!/usr/bin/env bash
```

Cuando el núcleo intenta ejecutar un fichero de texto con permiso x, lee los dos primeros bytes (#!). Al detectarlos sabe que no es código máquina, y usa la ruta que sigue para invocar el intérprete correspondiente, en este caso Bash, pasándole el fichero como argumento [5]. Hay cuatro formas de ejecutar un script y no son equivalentes: `./script.sh` (necesita x y el shebang), `bash script.sh` (no necesita x), `source script.sh` (se ejecuta en la Shell actual, sin proceso nuevo) y por nombre si está en `$PATH`. El capítulo siguiente vuelve sobre ellas.

8.3 Variables locales frente a variables de entorno exportadas

En Bash se crea una variable asignando sin espacios alrededor del igual (`MUESTRA="paciente_01"`). Hay una diferencia crucial de visibilidad:

- **Variable de Shell:** solo existe en la sesión que la creó. Un script o programa lanzado desde esa sesión no la verá.
- **Variable de entorno:** se declara con `export VARIABLE=valor` o, de forma equivalente, `declare -x VARIABLE`. El sistema la copia en el entorno de todos los procesos hijos lanzados a partir de ese momento.

Una variable de Bash ordinaria (`NOMBRE=valor`) es local a la sesión que la crea; solo se convierte en variable de entorno disponible para los procesos que esa sesión lance si se exporta con `export NOMBRE` o `export NOMBRE=valor`.

Se reproduce con `verification/chapter_results.sh export_makes_visible_to_subprocess`.

Resolución de la predicción 4.

Una variable asignada en Bash solo existe en la memoria del intérprete actual; los subprocessos no la reciben a menos que se exporte explícitamente.

8.4 Variables predefinidas y configuración permanente

El sistema y la Shell definen de forma automática variables fundamentales: `$HOME`, el directorio personal; `$PWD` y `$OLDPWD`, el directorio de trabajo actual y el anterior al último `cd`; `$PATH`, la lista de búsqueda; `$HOSTNAME`, el nombre de la máquina, útil para saber en qué nodo se ejecutó un análisis.

`$HOME`, `$PWD`, `$OLDPWD` y `$PATH` son variables de entorno ya definidas por el sistema: el directorio personal, el directorio de trabajo actual, el directorio de trabajo anterior tras el último `cd`, y la lista de búsqueda de ejecutables.

Se reproduce con `verification/chapter_results.sh oldpwd_tracks_previous_cd`.

Si desea que una variable esté disponible en las terminales que abra (por ejemplo, añadir una carpeta personal de herramientas con `export PATH="$HOME/bin:$PATH"`), añada esa línea a `~/.bashrc`, que leen las shells interactivas de Bash. Una shell de inicio de sesión puede leer en su lugar `~/.bash_profile`, y otras shells como `zsh` usan otros ficheros. Para recargar los cambios sin cerrar la terminal, `source ~/.bashrc`.

9 Ergonomía y productividad en la terminal

ESTUDIO Teclear letra por letra cada ruta larga genera fatiga y erratas. Bash incorpora la biblioteca GNU Readline, con atajos que conviene automatizar:

- **Tab:** autocompleta nombres. Si escribe `cd dat` y pulsa Tab, Bash completa `datos/`; dos pulsaciones muestran todas las alternativas. Elimina las erratas de tecleo en nombres largos; sigue siendo tarea suya comprobar que el nombre completado es el fichero que quería.
- **Ctrl+C:** envía SIGINT al proceso en primer plano. La mayoría de los programas terminan al recibirla, pero un programa puede atenderla de otra forma, retrasarla o ignorarla.
- **Ctrl+L:** limpia la pantalla, como `clear`.
- **Ctrl+A** y **Ctrl+E:** llevan el cursor al principio o al final de la línea.
- **Ctrl+U** y **Ctrl+K:** borran desde el cursor hasta el principio o hasta el final.
- **Alt+.**: inserta el último argumento de la orden anterior; útil para entrar en una carpeta recién creada.

Los atajos de GNU Readline en Bash, Tab para autocompletar, Ctrl+C para interrumpir, Ctrl+L para limpiar y Ctrl+A y Ctrl+E para saltar a los extremos de la línea, constituyen la ergonomía interactiva fundamental.

9.1 El historial y sus límites

Bash registra las órdenes en un historial que se guarda en `~/.bash_history`. Con Ctrl+R se busca en él de forma interactiva.

`history` refleja únicamente la sesión interactiva donde se ejecuta, tiene un límite de líneas conservadas y puede omitir órdenes ejecutadas en otra terminal; no es, por sí solo, un procedimiento documentado.

Un volcado del historial no sustituye a un protocolo: mezcla erratas, intentos fallidos y órdenes irrelevantes. Un procedimiento reproducible queda plasmado en un script limpio y un documento explicativo.

10 Espacio de almacenamiento y organización del proyecto

ESTUDIO Un proyecto de bioinformática genera con facilidad terabytes de ficheros temporales. Monitorizar el disco evita fallos durante ejecuciones que duran días. `df -h` muestra el espacio total, usado y disponible en cada sistema de archivos montado, en unidades legibles; `du -sh RUTA` calcula lo que ocupa un directorio con todo su contenido. Los discos externos y de red se incorporan al árbol con `mount` y se retiran con `umount`; en un clúster lo hace el administrador, pero `df` le mostrará qué está montado y dónde.

`df` inspecciona el espacio libre y ocupado en los sistemas de archivos montados, mientras que `du` calcula el espacio consumido por ficheros y directorios concretos.

Se reproduce con `verification/chapter_results.sh disk_usage_check`.

REFERENCIA En un sistema Linux moderno, los programas que corren de fondo, servicios como el planificador de trabajos del clúster o el servidor de ficheros, los gestiona `systemd`. Como usuario no los administra, pero dos órdenes le ahorran preguntas: `systemctl status NOMBRE` dice si un servicio está activo, y `journalctl -u NOMBRE` muestra su registro. Si un análisis falla porque «el disco de red no responde», es ahí donde está la explicación.

10.1 Estructura canónica de un proyecto bioinformático

Siguiendo las recomendaciones de buenas prácticas [3, 1], un proyecto mantiene una estructura de carpetas predecible: `data/raw/` con los datos originales, de solo lectura, que jamás se modifican; `data/processed/` con los datos limpios y transformados; `scripts/` con el código; `results/` con gráficos, tablas y resultados finales; y un `README.md` que explica el objetivo, las dependencias y cómo ejecutar el análisis.

11 Antes de una operación que sobrescribe o elimina

ESTUDIO La línea de órdenes no tiene papelera. Cuando se ejecuta `rm fichero`, el sistema desvincula el nombre de su inodo y los bloques quedan libres para reutilizarse. En la práctica, recuperar datos borrados no es fiable: actúe como si fuera irreversible. Para evitar accidentes, aplique siempre el protocolo de dos pasos:

1. **Verificar el directorio actual con `pwd`:** asegúrese de que está en la carpeta de trabajo correcta y no en la raíz o en su carpeta personal.
2. **Inspeccionar el objetivo exacto con `ls -ld -- RUTA`:** compruebe que la ruta señala al fichero o directorio que desea afectar antes de sustituir `ls -ld` por la orden destructiva.

Antes de cualquier operación que pueda sobrescribir o eliminar datos conviene confirmar el directorio con `pwd` e inspeccionar el objetivo exacto con `ls -ld -- RUTA` antes de ejecutar nada; solo se actúa sobre objetivos explícitos ya comprobados.

Un caso especialmente dañino aparece al usar variables no definidas dentro de órdenes recursivas: `rm -rf "$CARPETA_TEMPORAL"/*`, si la variable está vacía, se expande a `rm -rf "/"`, que intentará borrar todo aquello sobre lo que su usuario tenga permiso de escritura; en una cuenta de trabajo, todos sus datos, y con privilegios elevados, el sistema. El mecanismo de expansión de variables que lo explica se estudia en el capítulo

siguiente.

12 Reproducibilidad, empaquetado e integridad

ESTUDIO La reproducibilidad de un análisis exige documentar con precisión el contexto en el que se obtuvieron los resultados. Un registro reproducible recoge, como mínimo, siete campos:

Un registro mínimo de entorno reproducible incluye: sistema operativo, versión de Bash, origen y versión de los datos, fecha, las órdenes ejecutadas, el resultado esperado y las limitaciones conocidas.

Delimitación conceptual: reproducibilidad y replicabilidad.

En este libro, **reproducibilidad computacional** significa reejecutar el mismo procedimiento sobre los mismos datos brutos y el mismo entorno registrado y obtener el mismo resultado. El caso ideal es la coincidencia bit a bit; en flujos con paralelismo o aleatoriedad, una reproducción válida puede diferir en detalles no significativos, y lo que se exige es que el procedimiento y el criterio de comparación estén declarados. Obtener conclusiones convergentes con una cohorte independiente, nuevos datos o una implementación distinta es otra dimensión, la **replicabilidad**.

- *Reproducibilidad*: una investigadora ejecuta el script publicado sobre los datos originales y comprueba que la suma SHA-256 de la tabla de salida coincide con la del artículo.
- *Replicabilidad*: otro laboratorio recoge una nueva muestra de pacientes, aplica un protocolo análogo y confirma que el biomarcador mantiene significación.

En el anexo, `practice_assets/registrar_entorno.sh` genera un registro con estos siete campos a partir de `uname -a, bash --version y date`, y lo valida [3, pp. 67–70].

12.1 Empaquetado portable con tar

Para compartir un proyecto preservando su estructura y permisos se utiliza `tar` con compresión `gzip` [7]: `tar -czf entrega.tar.gz carpeta/` crea el paquete (`c` crear, `z` comprimir, `f` nombre del fichero); `tar -tzf entrega.tar.gz` lista el contenido sin extraerlo; `tar -xzf entrega.tar.gz` lo extrae.

Empaquetar el directorio `data/` con `tar -czf` y volver a extraerlo en un directorio distinto reproduce el mismo número de ficheros y las mismas sumas `sha256sum` que el original.

Se reproduce con `verification/check_package_roundtrip.sh`.

12.2 Integridad con sha256sum

Un fichero puede corromperse durante una descarga o por un fallo del soporte. Una función de resumen criptográfico como SHA-256 calcula una huella de 64 caracteres hexadecimales a partir de los bytes exactos del fichero. Si cambia un solo bit de un fichero de 50 GB, la huella cambia de forma impredecible, con altísima probabilidad en aproximadamente la mitad de sus bits.

`sha256sum -c` detecta, con probabilidad extremadamente alta, cualquier cambio de bytes respecto a un valor de referencia fiable; no distingue un cambio intencionado de una corrupción accidental, ni valida el significado del contenido, ni acredita autoría.

Se reproduce con `verification/check_manifest_integrity.sh`.

13 Método de verificación y errores frecuentes

ESENCIAL Ante cualquier instrucción propuesta por usted o por un asistente de inteligencia artificial, aplique el ciclo metodológico del curso:

El ciclo predecir, probar en pequeño, inspeccionar cada etapa, contrastar con la estructura esperada y registrar reduce la aceptación de una salida plausible pero incorrecta; este capítulo lo introduce y el siguiente lo recupera sin volver a enseñarlo.

1. **Predecir:** formule qué espera que haga la orden antes de pulsar Enter.
2. **Probar en pequeño:** ejecute primero sobre un subconjunto diminuto cuyo resultado conozca.
3. **Inspeccionar cada etapa:** compruebe el estado de salida (`echo $?`) y la salida intermedia.
4. **Contrastar con la estructura:** si esperaba 4 muestras, un resultado con 0 o con 400 es señal de alarma.
5. **Registrar:** guarde la orden validada en su script documentado.
6. **Validar antes de ejecutar:** un script se comprueba sin efectos con `bash -n script.sh`, que detecta errores de estructura, como comillas sin cerrar o un `if` sin `fi`, sin ejecutar ninguna orden.

Síntoma observable	Causa probable	Comprobación y corrección
No such file or directory command not found	directorio de trabajo incorrecto o errata en la ruta relativa orden no instalada o script local invocado sin <code>./</code>	<code>pwd</code> ; Tab para completar la ruta real <code>command -v ORDEN</code> ; anteponer <code>./</code>
Permission denied	falta <code>x</code> o <code>r</code>	<code>ls -l</code> ; <code>chmod u+x</code> solo sobre scripts propios revisados entrecorillar la ruta
Un fichero con espacios «desaparece» o parte la orden en dos	el espacio separa argumentos	
Una variable no llega a un script	definida como local y no exportada	<code>export NOMBRE</code> antes de lanzar el subproceso

Los cinco síntomas de esta tabla corresponden a los errores de principiante más frecuentes documentados en la presentación oficial de fundamentos de Linux del curso.

Compruebe antes de corregir: diagnóstico en dos fases.

Un guion de resumen devuelve `No such file or directory` en una terminal y `Permission denied` en otra. No aplique `chmod +x` de entrada. Primero `pwd` en ambas ventanas, para saber si están en el mismo directorio; después `ls -ld -- ./resultados/resumir.sh`, para confirmar que el fichero existe en esa ruta; y solo entonces `ls -l` para mirar sus permisos. Nunca corrija permisos sobre una ruta que no ha comprobado que apunta al fichero correcto.

14 Síntesis de referencia

Necesidad operativa	Construcción principal	Pregunta de control
Entender el diseño	todo es un fichero; herramientas modulares	¿Por qué combinamos herramientas pequeñas en vez de usar una sola?
Orientarse	<code>pwd</code> , <code>ls -la</code> , <code>cd</code>	¿Desde qué directorio de trabajo se interpreta esta ruta?
Organizar	<code>mkdir -p</code> , <code>cp</code> , <code>mv</code> , <code>touch</code>	¿He comprobado el destino antes de escribir?
Inspeccionar sin alterar	<code>file</code> , <code>head</code> , <code>tail</code> , <code>less</code> , <code>wc -l</code>	¿Qué unidad estoy contando y cómo salgo (q)?
Gestionar enlaces	<code>ln -s ORIGEN ENLACE</code>	¿Estoy evitando duplicar gigabytes en el disco?
Proteger y compartir	<code>rxw</code> , <code>chmod</code> , <code>chown</code>	¿Quién puede leer, escribir o ejecutar este fichero?
Ejecutar programas	<code>\$PATH</code> , <code>./</code> , <code>\$?</code>	¿Qué significa el estado de salida para este programa concreto?

Necesidad operativa	Construcción principal	Pregunta de control
Configurar el entorno	variables locales frente a <code>export</code> , <code>~/ .bashrc</code>	¿Esta variable llega a los subprocesos que lance?
Ergonomía	Tab, Ctrl+C, Ctrl+L, Ctrl+R	¿Estoy usando el autocompletado para evitar erratas?
Monitorizar disco	<code>df -h</code> , <code>du -sh</code>	¿Hay espacio suficiente antes de lanzar el análisis?
Seguridad destructiva	<code>pwd</code> y <code>ls -ld -- RUTA</code>	¿Es exactamente el objetivo previsto?
Reproducir y entregar	<code>tar -czf</code> , <code>tar -tzf</code> , <code>sha256sum -c</code>	¿El paquete conserva la estructura y los bytes?

La tabla de síntesis enlaza cada necesidad operativa del capítulo con su construcción principal y una pregunta de control verificable.

15 Glosario esencial

- **Unix / tipo Unix:** sistema operativo original de 1969 frente a sus descendientes y clones (Linux, macOS, BSD).
- **Proceso:** programa en ejecución con un identificador único (PID) y recursos propios.
- **Terminal:** interfaz de texto que conecta al usuario con el intérprete de órdenes.
- **Shell:** intérprete de órdenes; en este texto, Bash.
- **Estado de salida** (\$?): entero que devuelve un proceso al terminar; 0 cuando no detectó error, otro valor para una condición que el programa define, a menudo un error.
- **\$PATH:** lista ordenada de directorios donde la Shell busca ejecutables por nombre.
- **Directorio de trabajo:** posición actual en el árbol desde donde se resuelven las rutas relativas.
- **Enlace simbólico:** fichero especial que apunta a otra ruta sin duplicar los datos.
- **Permisos rwx:** lectura, escritura y ejecución para usuario, grupo y otros.
- **Variable de entorno:** variable exportada, disponible para los subprocesos de la Shell.
- **Reproducibilidad:** capacidad de repetir un análisis obteniendo el mismo resultado según el criterio de comparación declarado, gracias al registro explícito del entorno.
- **Suma SHA-256:** huella criptográfica que detecta con altísima probabilidad cualquier cambio de bytes en un fichero.

16 Fuentes y límites de uso

Este capítulo se apoya en las presentaciones docentes del curso, 20250909_intro.pptx (historia, arquitectura y filosofía Unix) y 20250911_fundamentos.pptx (modelo de procesos, permisos, enlaces, variables de entorno, órdenes fundamentales y ergonomía), y en la bibliografía de referencia: la guía de bolsillo de Barrett [2], bibliografía básica de la asignatura; Buffalo [3] y Allesina y Wilmes [1] para la organización reproducible del trabajo; y los manuales de GNU Bash [5], GNU Coreutils [6] y GNU tar [7] para el comportamiento exacto de cada orden. Las opciones y mensajes descritos corresponden a las versiones de GNU instaladas en un Linux actual; en macOS y BSD pueden diferir.

Bibliografía

- [1] Stefano Allesina and Madlen Wilmes. *Computing Skills for Biologists: A Toolbox*. Princeton University Press, 2019. Ejemplos de línea de comandos contrastados en el PDF local; no sostienen ninguna afirmación en solitario.
- [2] Daniel J. Barrett. *Linux Pocket Guide: Essential Commands*. O'Reilly Media, 4 edition, 2024. Bibliografía básica de la asignatura según la presentación de introducción del curso.

- [3] Vince Buffalo. *Bioinformatics Data Skills: Reproducible and Robust Research with Open Source Tools*. O'Reilly Media, 2015. Pasajes comprobados en las páginas físicas 51–54 y 67–70 del PDF local.
- [4] Canonical. *The Linux command line for beginners*, 2026. Documentación oficial. Consultada el 6 de agosto de 2026. <https://documentation.ubuntu.com/desktop/en/latest/tutorial/the-linux-command-line-for-beginners/>.
- [5] GNU Project. *Bash Reference Manual*, 2026. Consultado el 6 de agosto de 2026. <https://www.gnu.org/software/bash/manual/bash.pdf>.
- [6] GNU Project. *GNU Coreutils Manual*, 2026. Documentación de pwd, ls, cd, mkdir, cp, mv, head, tail, wc y file. Consultada el 6 de agosto de 2026. <https://www.gnu.org/software/coreutils/manual/coreutils.html>.
- [7] GNU Project. *GNU tar Manual*, 2026. Consultado el 6 de agosto de 2026. <https://www.gnu.org/software/tar/manual/tar.html>.